oriented programming style, state is managed by keeping it contained to individual objects. In a functional programming style, the state is essentially managed by not putting it explicitly in the program *at all*.

Functional programming and pure functions

A function that always returns the same value for the same arguments and that has no side effects is called a *pure function*. Because it's very easy to create a non-pure function in CoffeeScript, it's debatable to what extent it can be called a functional programming language. But first-class functions in CoffeeScript mean that, at the least, CoffeeScript supports functional-style programming.

State can't always be avoided; there's still state such as in a database or user interface. You can't keep track of how many users have visited your website unless there's state somewhere. Users can't interact with a completely stateless interface. State exists and your program has to deal with it. The question is, exactly when is state necessary and where should it go?

6.2.4. World state

"Wait a minute," says Scruffy. "It's all well and good to take unnecessary state out of the program, but the real world has state! Without state there's no way to sell cameras because there's no stock count." That's right, Scruffy. There has to be state somewhere; the key is to isolate it and not to share it. In this case, suppose that the stock level is kept in a database. Databases are good at dealing with state. The values in the database are retrieved via callbacks:

```
db.stock 'ijuf', (error, response) ->
  # handling code here
```

This database call could be on the client side or server side. It doesn't matter.